Maneuver Prediction from Vehicle Camera Images

DATASCI 281 · Group 2 · Dawson, Matt, Nithya, Victor

Abstract

We predict a vehicle's upcoming maneuver (straight, left-turn, or right-turn) from camera imagery alone, using the Waymo Open Dataset End-to-End (WOD-E2E). Because a maneuver is fundamentally a motion event, we test a central hypothesis: features that encode how a scene changes over time should outperform features describing a single frame's context. We build and benchmark four feature families: cheap classical descriptors (HOG, HSV histograms), geometry-aware summaries (road geometry, YOLO object positions, vehicle occupancy), a hand-crafted temporal motion family (22-dimensional Trend + Farneback optical flow), and learned deep representations (2D CNN, prior-only 3D CNN, and frozen MoViNet video embeddings) across logistic regression, RBF-SVM, and random-forest classifiers on a single frozen, stratified split. Labels are derived directly from future-trajectory geometry via a port of Waymo's ClassifyTrack, and we restrict the original 5-class task to 3 classes after establishing that lane changes have no present-frame signature and that their world-coordinate labels conflate genuine lane changes with in-lane curve-following. The results support the hypothesis decisively: the 22 hand-crafted temporal dimensions (0.671 accuracy / 0.607 macro-F1) beat every single-frame family, classical or learned, while appearance-only features, including a driving-pretrained BDD100K ResNet-50, cluster near the 0.461 majority baseline. Fusing explicit motion with a frozen MoViNet video embedding under an SVM reaches our best result of 0.746 accuracy / 0.693 macro-F1, showing that hand-crafted and learned motion are complementary. Motion, not appearance or domain pretraining, is the signal.

1. Introduction

In the context of self driving, predicting a vehicle's upcoming maneuver is a crucial part of the puzzle. Being able to determine which direction is the intended vector of travel is necessary pre-requisite for the downstream steps of plotting out a trajectory and coming up with the control inputs required to achieve that movement.

While modern end-to-end driving models increasingly rely on complex, multi-sensor arrays (including LiDAR and radar), extracting motion and intent cues from camera data alone remains a highly desirable capability for system redundancy. This task is inherently difficult because a maneuver unfolds over time; a single static frame often lacks the context required to distinguish between maintaining a lane and initiating a turn.

For our project, we leverage the Waymo Open Dataset End-to-End (WOD-E2E) benchmark to test whether or not features that are reliant on a single frame can provide enough information to accurately predict intended maneuver, as well as how well can temporal data in a short window before. This project is focused on predicting 3 total types of maneuvers, straight, left turn, right turn. This was derived from a subset of 5 total maneuvers, which included left lane change and right lane change. However it was decided that it was not possible to reliably create labels for these two classes so these were dropped.

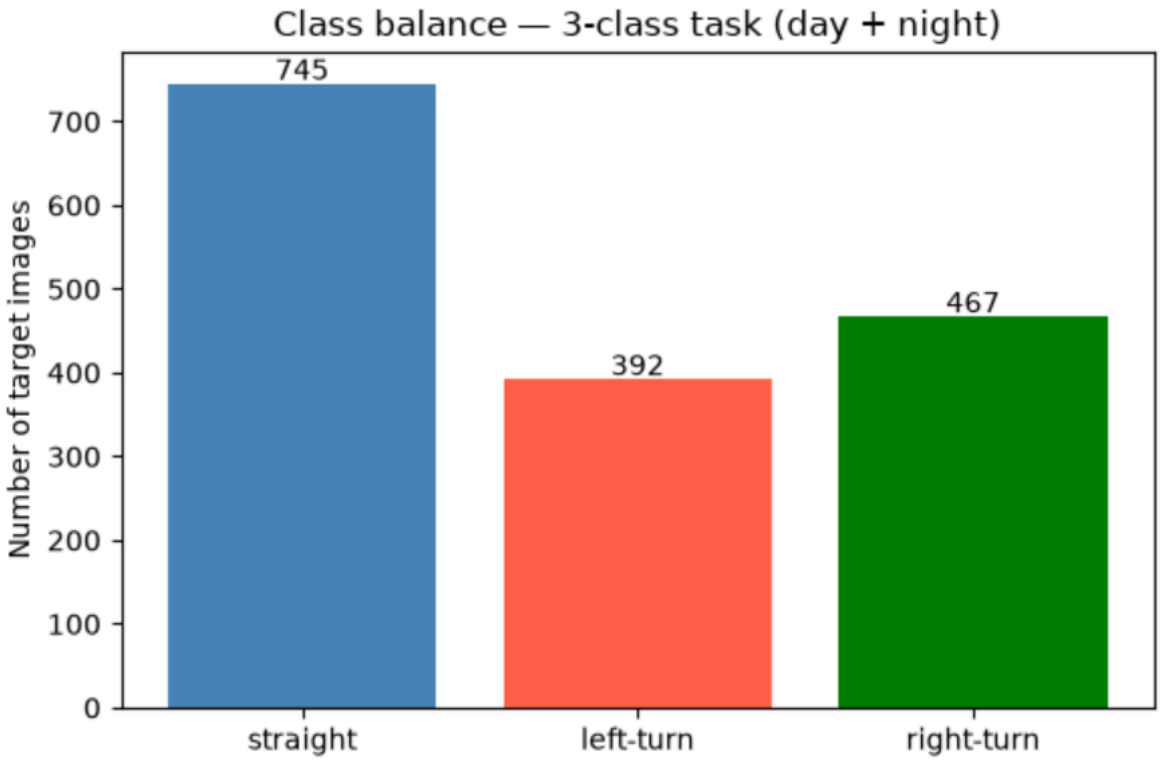

Figure 0: Distribution of 3 movement classes

2. Data and Preprocessing

2.1 Dataset and Preprocessing

We use the Waymo Open Dataset End-to-End (WOD-E2E) [Xu et al.], a camera-only end-to-end driving benchmark. Each sequence provides synchronized imagery from eight vehicle-mounted cameras together with the vehicle's future trajectory (a 5-second horizon of 20 waypoints sampled at 4 Hz in an vehicle-centered coordinate frame), plus recorded past states and a high-level routing intent field. Of these, only the imagery is ever visible to our models: the future trajectory serves exclusively as the supervision source for the labels of Section 2.2, the intent field appears only as a labeling comparison baseline (Section 2.2), and the past states are unused. We work from Waymo's training split (2,037 sequences) and validation split (479

sequences), which we pool and re-partition into our own train, validation, and test folds (Section 2.3). The official test split withholds future trajectories and therefore cannot be labeled for our task, a constraint that shapes the evaluation design in Section 2.3.

Raw sequences are processed with the StandardE2E pipeline, which stitches the eight camera views into a single 142×384×3 panoramic frame per timestep, crops the hood region, and writes one file per frame, for a total of 415,663 frames across the training split. A committed manifest ties the dataset together. For each sequence it records the maneuver label, a single target frame used for single-frame feature extraction, and a context window of up to 20 preceding frames (sampled 250 ms apart; mean 12.0 frames, with five sequences retaining only one) used by the temporal features of Section 3. The target frame is the frame immediately preceding the maneuver-defining frame, so every feature describes the scene before the maneuver unfolds.

Two properties of the panoramic format matter downstream. First, classical perspective-geometry operations such as lane detection and vanishing-point estimation assume a single-camera perspective projection and are ill-posed on a curved multi-camera stitch. On full panoramas, lane detection failed even on clean daytime frames, while the same detector applied to the central single-camera crop (columns 128 to 256) recovered lane lines converging to a stable vanishing point. Road-geometry features therefore operate on the central crop rather than the full panorama. Second, the stitch is provided pre-rectified by StandardE2E (hood removed, exposures blended), so no additional preprocessing was applied; features that require scaling are standardized downstream with statistics fit on the reshuffled training fold only (Section 4).

Figure 1: A 142×384 eight-camera panoramic frame with the central single-camera crop (columns 128–256, cyan) outlined. Perspective-geometry features assume a single-camera projection and fail on the curved stitch; lane detection recovers on the central crop, so all road-geometry features operate there.

2.2 Maneuver Labeling

Each sequence receives one label derived from the geometry of the vehicle's future trajectory, using a port of ClassifyTrack, the trajectory-shape classifier the Waymo Open Motion Dataset

uses to categorize vehicle tracks [Ettinger et al.], with Waymo's thresholds adopted unchanged. A trajectory whose net heading change reaches 30° is a turn. Below that threshold, a net lateral displacement of at least 2.5 m marks a lane change; otherwise the sequence is straight. Turn and lane-change direction follows the sign of the net lateral displacement in the vehicle's own coordinate frame (positive y to the left). Near-stationary sequences (maximum speed below 2.0 m/s with net displacement under 3.0 m; 11 sequences, 0.5%) are excluded.

Two adaptations were required. First, the WOD-E2E future states populate only position and timestamp, so heading is reconstructed from the trajectory tangent and speed from consecutive displacements over the timestamp interval. Heading reconstruction is gated to trajectory segments spanning at least 0.5 m of displacement, to avoid ill-defined headings. Second, because every frame in a sequence carries its own future window, frame-level classifications are reduced to a sequence label by taking the highest-ranked maneuver observed (turn > lane-change > straight > stationary), so a clip containing a turn is labeled a turn even when most of its frames are straight. In the case of multiple maneuvers, the first maneuver is selected.

This formulation replaced an earlier ad-hoc labeler that inferred turn direction from the angle between the two halves of the trajectory. Auditing the replacement exposed the value of adopting the standard method: the old heuristic had flipped the left/right labels of 26 sequences whose trajectories contained near-stationary segments, where a half-trajectory's direction is numerically undefined.

We deliberately do not use WOD-E2E's native intent field. Its values (GO_STRAIGHT, GO_LEFT, GO_RIGHT) are routing commands that describe where the vehicle is headed on the road network, not necessarily any observable maneuver executed in the observation window. A cross-tabulation on a 200-sequence sample makes the mismatch concrete: only 63.6% of GO_LEFT sequences execute a left turn, 35.0% of GO_STRAIGHT sequences turn, and no intent value maps to lane changes at all.

2.3 Task Restriction and Data Splits

The project began as a 5-class task (straight, left-turn, right-turn, lane-change-left, lane-change-right, with 745 / 392 / 467 / 200 / 222 sequences respectively). The lane-change classes proved unpredictable from the vehicle camera imagery. The decisive problem emerged from inspecting labeled sequences directly: in multiple cases the vehicle stayed in its lane while the road curved gently, accumulating more than 2.5 m of lateral displacement with a heading change below the 30° turn threshold, which is exactly the trajectory geometry the labeler defines as a lane change. Because labels are derived in world coordinates and the dataset provides no lane-relative positioning, these curve-following sequences are impossible to distinguish from true lane changes, so the lane-change class irreducibly mixes the two behaviors. Prediction results corroborated the problem: across five independent feature approaches, recall on labeled lane changes ranged from 0.00 to 0.12, consistent with chance, and even lateral moves of 5 m or more were missed in all 11 cases. We therefore dropped the lane-change sequences rather than folding them into straight, which would have mislabeled roughly 400 genuine lateral maneuvers.

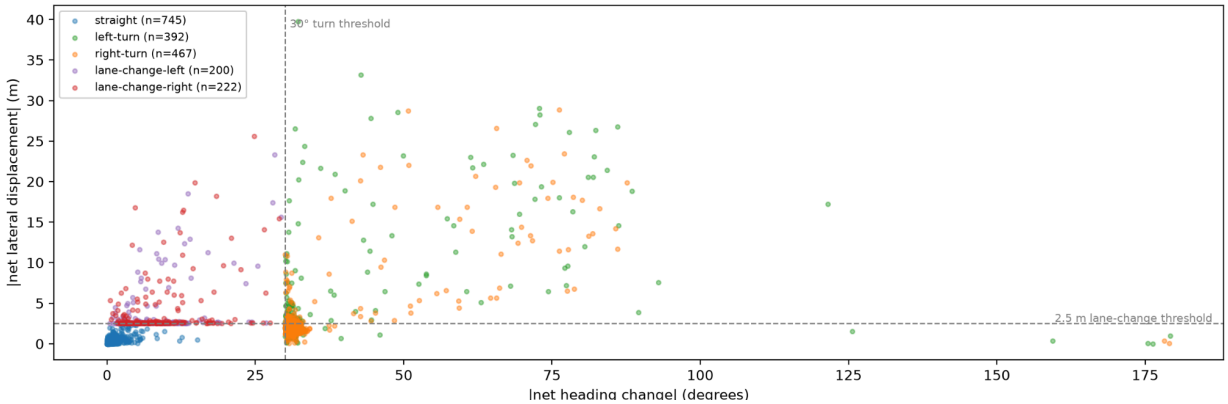

Figure 2: Net heading change vs. net lateral displacement of each sequence's label-defining trajectory (n=2,026), with the ClassifyTrack thresholds dashed.

Because the official test split cannot be labeled, we pooled the labeled training and validation splits and re-split them ourselves. After the 3-class restriction the pool contains 1,966 sequences (straight 906, right-turn 589, left-turn 471), divided into train, validation, and test folds of 1,376, 295, and 295 by a stratified shuffle with a fixed seed. The fold assignment is frozen in a committed file and every feature family joins it by sequence id, so all models in this paper, classical, temporal, and learned alike, are trained and evaluated on identical folds. Figure 3 shows the class balance across the three folds.

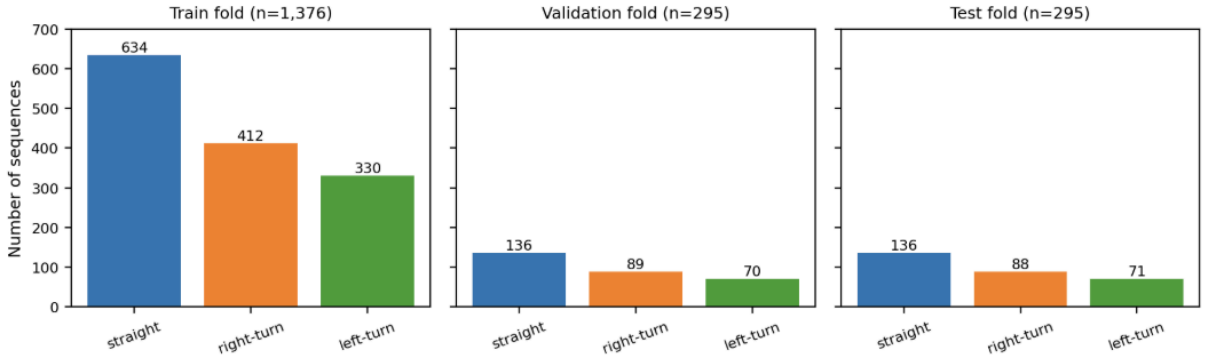

Figure 3 - Class Balance by Dataset Split

3. Features

We designed a range of features spanning cheap classical descriptors, geometry-aware summaries, temporal motion features, and learned deep representations. A vehicle maneuver is fundamentally a motion event, so our central hypothesis was that features encoding how the

scene changes over time would outperform features describing a single frame's context. We created examples of both to test this hypothesis directly.

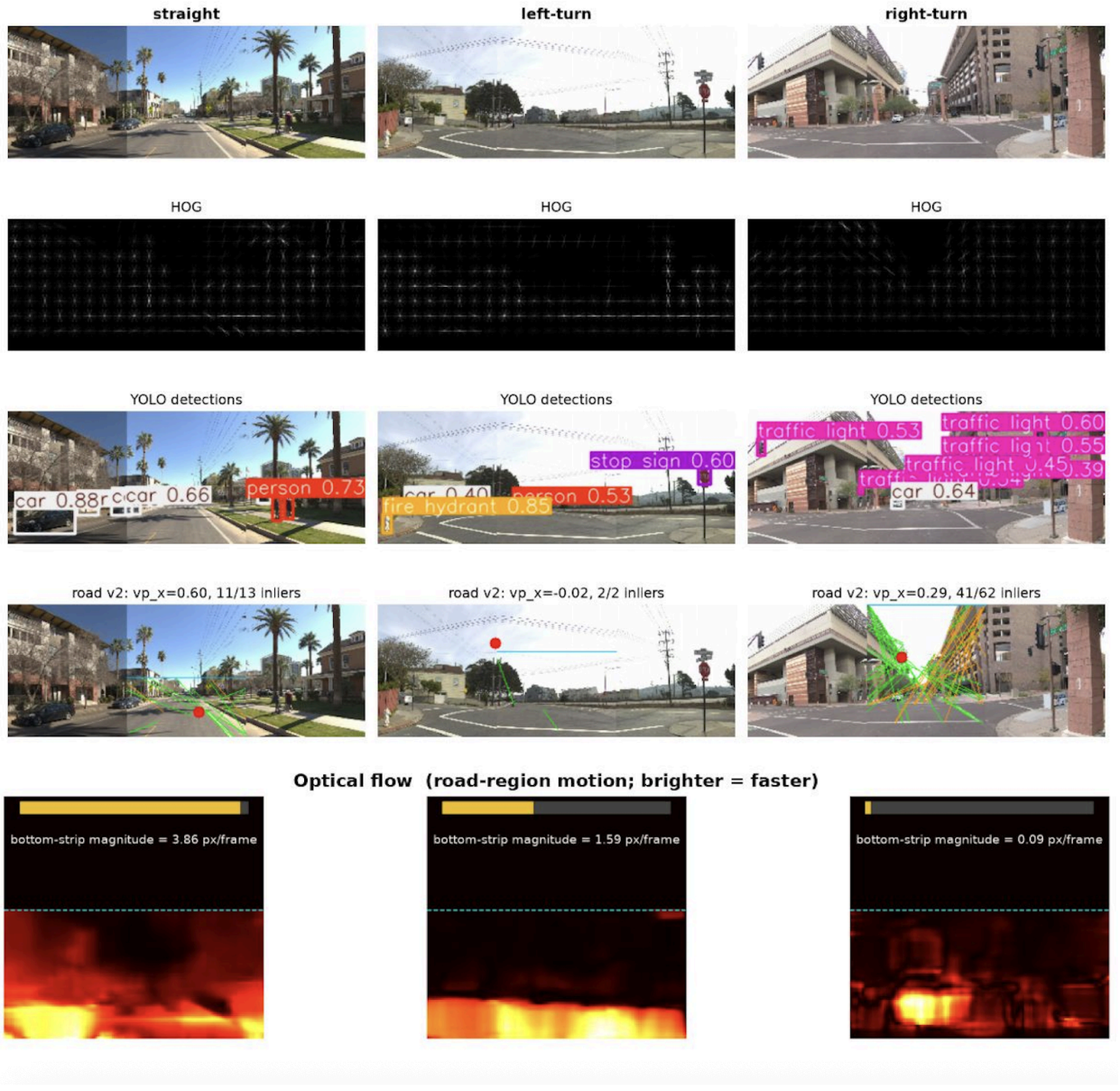

Figure 4: Each feature family visualized on one example frame per class (columns: straight, left-turn, right-turn). Rows: raw central crop, HOG gradients, YOLO detections, road-geometry vanishing-point fit (red dot, inlier lines), and optical-flow magnitude over the road region (brighter = faster; bottom-strip magnitude annotated).

Simple (classical) features

HOG (5,796 dimensions) [Dalal and Triggs] captures edge and gradient structure such as lane lines, road boundaries, and the horizon. HSV color histograms (34 dimensions) summarize scene palette and lighting. Both are single-frame and inexpensive. We expect color in

particular to be weak here, since it describes the scene's appearance rather than whether the vehicle is turning, and we test that in the results.

After performing PCA and t-SNE see some further evidence for our expectation that this ends up being a fairly weak predictor, as we do not see strong separability between the features and the manoeuvers. It does appear we may have some weak clustering for straight vs both classes of turns

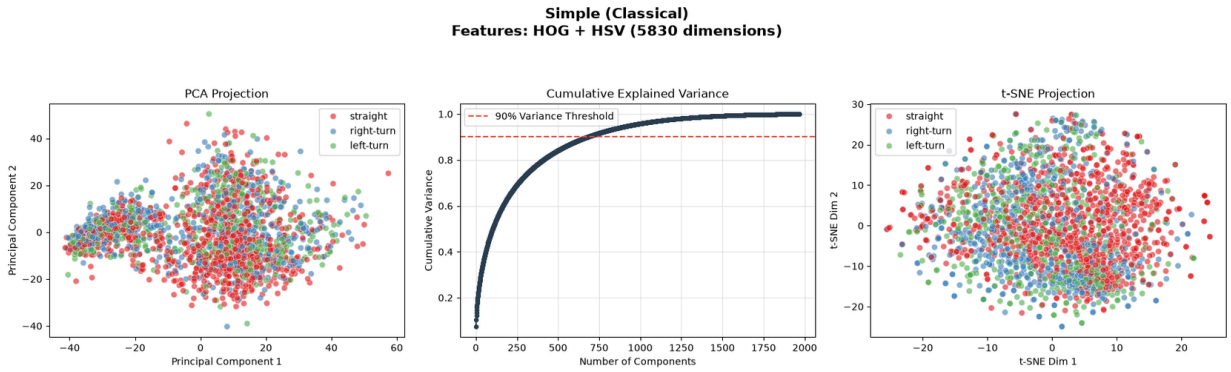

Figure 5: PCA + t-SNE for simple features

Complex features

YOLO detections [Jocer et al.] with relative position (32 dimensions) locate other scene objects. Road geometry (10 dimensions) summarizes Hough line orientations, the vanishing point, and the road's centroid. Vehicle occupancy (7 dimensions) is a coarse spatial grid of detected additional vehicles in the frame. Trend and Optical Flow (22 dimensions: 10 trend, 12 Farneback optical flow [Farneback], computed with OpenCV [Bradski]) is our temporal family, encoding how scene statistics and pixel motion evolve across the sequence, and is the family most directly aligned with classifying the future maneuver of the vehicle.

After performing dimensional analysis it seems like we still do not have strong clustering for these complex spatial features as well.

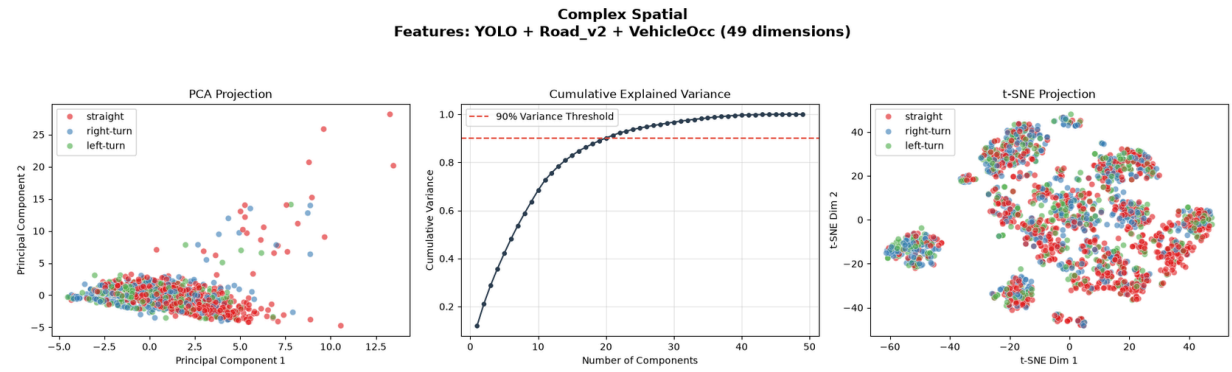

Figure 6: PCA + t-SNE for Complex Spatial features

Learned features

To compare engineered descriptors with learned appearance representations, we trained a compact 2D CNN on the single RGB target frame, resized to 96 x 256 pixels. Its convolutional layers learn spatial structure and the penultimate dense activation supplies the feature vector used by downstream classifiers. We also include a frozen ResNet-50 [He et al.] pretrained on BDD100K [Yu et al.] as a domain-transfer control. Both representations see only one frame, so their predictive power must come from scene layout and appearance rather than motion.

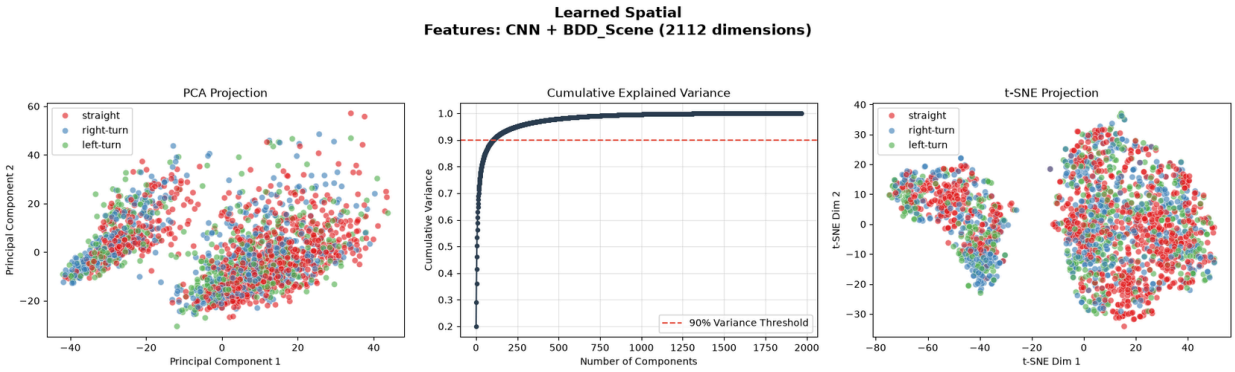

Figure 7: PCA + t-SNE for the learned spatial features, custom CNN + pre-trained driving based CNN

To expose motion while preserving causality, the custom 3D CNN consumes a 16-frame RGB clip sampled chronologically from the 20 frames immediately preceding the target. When fewer than 16 legal prior frames are available, the earliest available frame is repeated; a sequence with no prior history receives black padding. The network uses spatiotemporal convolutions and exports its penultimate dense activation. The target frame and every later frame are strictly excluded, preventing future information from leaking into the representation.

Our transfer-learning model applies the same prior-only policy at 96 x 256 resolution with a frozen MoViNet-A0-Stream backbone [Kondratyuk et al] pretrained on Kinetics-600 [Carreira et al.]. Global average pooling produces a 480-dimensional video embedding. A regularized dense head is tuned on an internal 15% partition of the training fold and then refit on the complete training fold for the selected number of epochs; its penultimate activation is cached as the learned video feature. We do not use horizontal flips because they would interchange left and right turns.

After conducting dimensional analysis for temporal motion, we can see that we have some evidence of strong clustering. Specifically we see that the straight maneuvers are clustered pretty tightly. The right and left turns do not appear to be clearly split

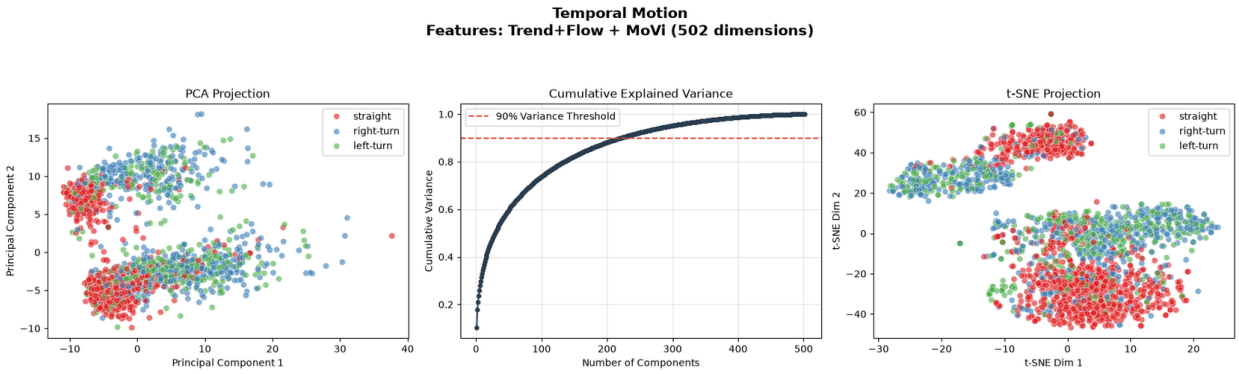

Figure 8: PCA + t-SNE for temporal data, Trend+Flow and also MoViNet

Dimensionality Analysis (PCA + t-SNE)

To understand how these diverse dimensions interact, we projected the complete feature stack using both PCA and t-SNE [van der Maaten and Hinton]. Looking at the results (Figure 8) we can see that when we account for all factors, we lose the clustering we could observe with the temporal data, as it is likely masked by the high dimensionality of the other features. Looking at the cumulative explained variance however, we can see that we still need a fairly high number, approximately 750, of the principal components, in order to explain 90% of the variance. This suggests that a large subset of these combined features is required to capture the highly complex representation of the driving environment, and as a consequence may surface information valuable for classifying maneuvers.

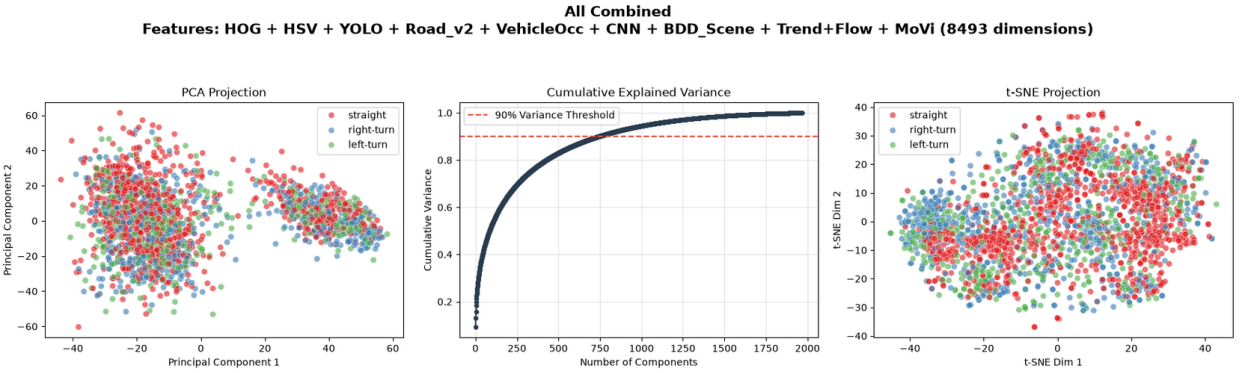

Figure 9: PCA + t-SNE for all factors combined

4. Classification

We evaluated three classifiers spanning linear and non-linear families: Logistic regression (linear, cross entropy loss), a Support Vector Machine with an RBF kernel (non-linear margin, hinge loss), and a Random Forest (non-linear ensemble of Gini-split trees) implemented in skikit-learn [Pedregosa et al.]. This set covers a linear baseline, a kernel method suited to moderate-dimensional continuous features, and a tree ensemble that is robust to feature scaling and mixed feature types. All three use class_weight='balanced' to counter the class imbalance

(straight is the majority at roughly 46 percent), and we report macro-F1 as the primary metric so the rarer turn cases count equally. The majority-class baseline is 0.461 accuracy (macro-F1 0.210); uniform random guessing would score 0.322

We use a single frozen split of the 1,966 pooled sequences into train (1,376), validation (295), and test (295), stratified by class. Every feature family is joined to this split by sequence id, so all models are trained and scored on identical folds. We fit each model on train, select hyperparameters on validation, and report final numbers once on the held-out test set. Features that require scaling (SVM, Logistic Regression) are standardized with statistics fit on train only; Random Forests use raw features, since trees are scale-invariant.

Hyperparameter search: we grid-searched each classifier on the validation data: SVM over C and gamma, Random Forest over number of trees and maximum depth, and Logistic Regression over C. Searching on validation and never on test keeps the test set a clean estimate of generalization and avoids overfitting model selection. One caveat we handle explicitly: the learned CNN features were trained with early stopping on the validation fold, so validation overstates them; we therefore treat CNN combinations as an ablation and select the final model among the leakage-clean feature sets.

Deep-model training and leakage controls
The 2D and from-scratch 3D networks use Adam with sparse categorical cross-entropy and tune capacity, kernel size, hidden width, dropout, L2 regularization, learning rate, and batch size. Those early experiments used the shared validation fold for early stopping, so their embeddings and feature stacks are reported as ablations rather than leakage-clean model-selection candidates. The MoViNet workflow corrects this issue: backbone weights remain frozen, a stratified 15% subset of the training fold selects the dense head and epoch count, and the selected head is refit on all training examples without using the shared validation or test folds for early stopping.

Held-out test results

Pipeline	Val Macro-F1	Test Accuracy	Test Macro-F1	Primary signal
Majority baseline (always straight)	0.210	0.461	0.210	No predictive signal
BDD100K ResNet-50	0.522	0.512	0.441	Single-frame domain appearance
2D CNN	0.597	0.563	0.516	Single-frame learned appearance
Prior-only 3D CNN	0.606	0.624	0.563	Motion learned from scratch

Frozen MoViNet head	0.610	0.675	0.599	Kinetics-pretrained video
Trend+Flow / Random Forest	0.611	0.671	0.607	Explicit temporal motion
Trend+Flow + MoViNet / SVM	**0.606**	**0.746**	**0.693**	**Fused explicit + learned motion**

Table 1: Validation macro-F1 and held-out test performance for the principal baselines and temporal models. Validation is used for model selection (Section 4); test values are computed once on the same 295-sequence fold.

The temporal advantage is consistent across representations. Trend+Flow with a random forest, despite using only 22 engineered dimensions, outperforms every single-frame pipeline. A frozen MoViNet head raises accuracy further, and the combination of explicit Trend+Flow and learned MoViNet motion reaches 0.746 accuracy and 0.693 macro-F1. Relative to Trend+Flow alone, the fused model gains 0.075 accuracy and 0.086 macro-F1, indicating that the two temporal feature families are complementary rather than redundant.

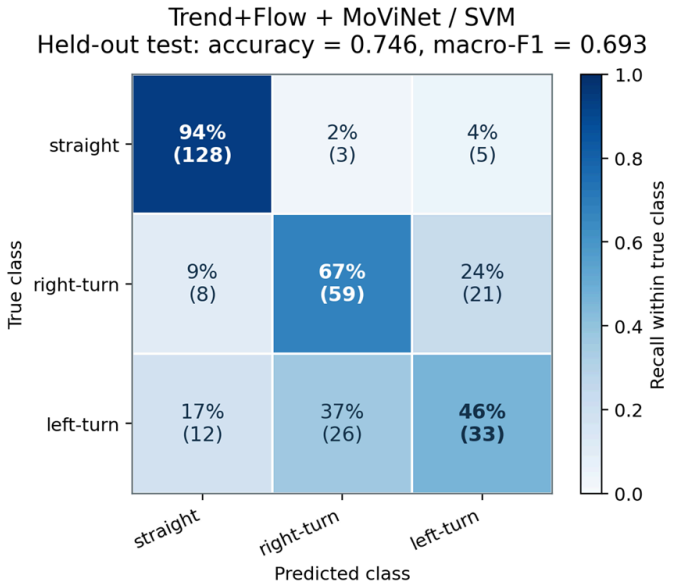

Figure 10: Row-normalized confusion matrix for the selected Trend+Flow + MoViNet RBF-SVM. Each cell shows recall percentage with count in parentheses.

The fused model recalls 94% of straight sequences, 67% of right turns, and 46% of left turns. Most residual errors confuse the two turn directions: 37% of left turns are predicted as right turns and 24% of right turns are predicted as left turns. One plausible explanation is road position in right-hand traffic: a vehicle preparing to turn left may remain near the road center and resemble straight travel, while a right-turn approach more often carries curb-side cues. This interpretation remains a hypothesis because road topology and lane-relative position are not explicitly annotated.

5. Generalizability

To ensure our models generalize to unseen driving data, we confined all hyperparameter tuning exclusively to the validation set. By doing so, the test set was kept unseen and did not influence our choice of hyper parameters. Since each input was also from a unique sequence from the data, there is little risk of data leakage from our test set into either the train or validation data set.

The primary risk to generalizability in our pipeline emerged from the deep learning models. Because the learned CNN features utilized early stopping based on validation loss, their performance metrics were inherently subject to data leakage, making them look artificially strong on the validation split. To maintain a clean, statistically sound evaluation, we treated the CNN configurations as ablations rather than primary candidates during model selection. The resulting test metrics closely mirrored the validation metrics, confirming that robust generalization was achieved.

6. Efficiency vs. Accuracy

Efficiency must be separated into two stages: one-time feature extraction and repeated downstream classifier fitting. The standalone 2D CNN required 321 seconds of model training and reached 0.516 macro-F1; the custom prior-only 3D CNN required 1,606 seconds and reached 0.563. Frozen MoViNet extraction is also costly on a CPU, but the video embeddings can be computed once and cached. Figure 10 therefore compares the tuned SVM and random-forest classifiers after all feature vectors are available. Classifier training and inference times were measured in single runs on an Apple M3 Pro (CPU only); the 2D and 3D CNN training times were measured on an AMD Ryzen 4500U with Radeon Graphics (CPU only).

The accuracy-oriented choice, Trend+Flow + MoViNet with an RBF-SVM, trains on cached features in 0.20 seconds and predicts a sequence in 0.23 ms while achieving 0.693 macro-F1. The efficiency-oriented choice, the 22-dimensional Trend+Flow random forest, reaches 0.607 macro-F1 and 0.671 accuracy, retaining 87.6% of the best macro-F1 without requiring a deep video embedding. Thus the fused model is preferred when offline extraction and caching are acceptable, whereas Trend+Flow alone offers the stronger end-to-end efficiency tradeoff. The most accurate downstream classifier is the cheapest to fit after caching, but it is not the cheapest complete feature pipeline.

.

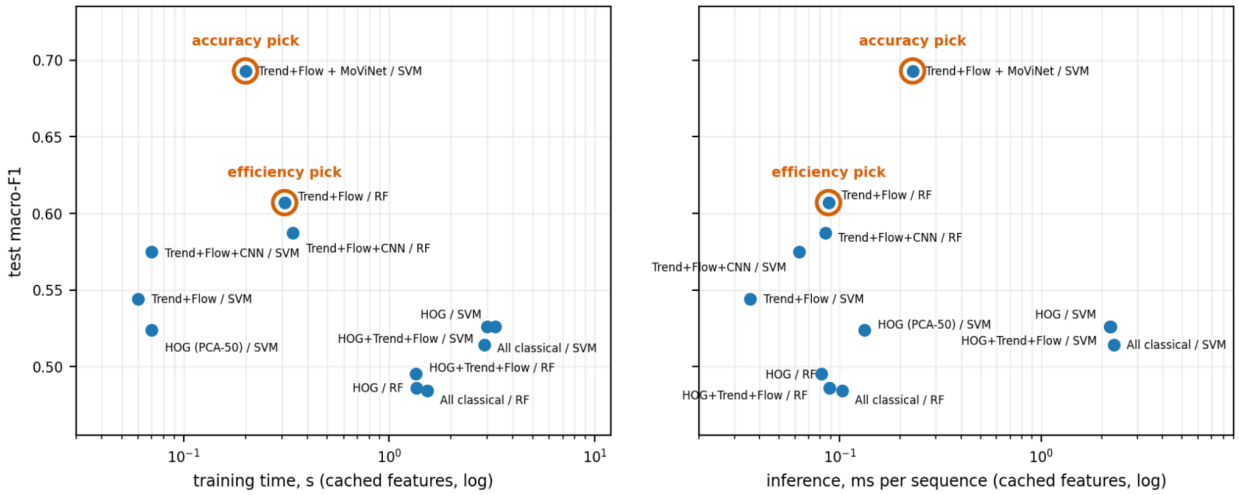

Figure 11: Test macro-F1 versus model training time (left) and per-sequence classifier inference time (right), log scales, tuned SVM/RF configurations on cached features (logistic-regression rows omitted as dominated). The accuracy pick reaches 0.693 while training in 0.20 s and predicting in 0.23 ms; the efficiency pick retains 88% of that macro-F1 at 22 dimensions. One-time feature-extraction costs are reported in the text.

7. Conclusion and Future Work

This project set out to predict ego-vehicle maneuvers from camera imagery and, in doing so, to test whether temporal features outperform single-frame ones. The evidence shows that motion is the signal. Our 22 hand-crafted temporal dimensions (Trend + Optical Flow) reached 0.671 accuracy and 0.607 macro-F1, beating every single-frame family (classical or learned) none of which exceeded 0.576 accuracy. Appearance-only features fared poorly: HSV color histograms landed near the 0.461 majority baseline, confirming that scene palette carries essentially no maneuver signal, and even a driving-pretrained single-frame embedding (BDD100K ResNet-50) fell into the appearance cluster and *cost* 0.14 macro-F1 when stacked. Domain pretraining is not a substitute for the temporal dimension.

Learned and hand-crafted motion proved complementary rather than redundant. Fusing Trend+Flow with a frozen MoViNet video embedding under an RBF-SVM produced our best model: 0.746 accuracy and 0.693 macro-F1, a gain of +0.086 macro-F1 over the classical best. This pipeline was also, after one-time feature caching, the cheapest downstream model to train (0.20 s), illustrating that the most accurate combination need not be the most expensive to fit. However, it is not the cheapest end-to-end feature pipeline, since the MoViNet extraction is costly upfront.

Two methodological findings were as important as the predictive ones. First, the 3-class restriction was forced by the data, not chosen for convenience: without lane-relative positioning, world-coordinate labels cannot separate a lane change from a car following a curving road, and

the lane-change class showed no present-frame signature across five independent null tests. Second, validation selection is noisy at n=295 (an unrestricted search picked a feature stack that collapsed from 0.623 validation to 0.510 test accuracy) so we constrained selection to compact hand-crafted combinations and treated frozen embeddings as ablations. Across every model, left-turn remained the hardest class (recall 0.25–0.46); as the rarest class with the least prior context, this reflects a class asymmetry in the data rather than a modeling choice.

Several directions would extend this work. A single 70/15/15 split leaves metric variance unquantified; repeated seeds or grouped cross-validation would address it. More labeled data with class-balanced augmentation would reduce the overfitting risk that deep video models face at this scale, and GPU-based extraction would replace the CPU-only video pipeline that currently bottlenecks iteration. The most compelling extension applies the same pipeline to *other* observed vehicles rather than the ego-vehicle (tracking and cropping each agent, computing ego-motion-compensated flow, and drawing ClassifyTrack labels from WOMD agent tracks) since anticipating the maneuvers of other road users from pixels alone, without their sensor or lidar data, is where camera-only maneuver prediction matters most.

Group Contributions

Dawson: Data pipeline & ClassifyTrack labeling (incl. label QA), Classical + Trend+Flow features, Pooled benchmark & selection protocol, Efficiency vs. Accuracy analysis, Sections 2, 5, 6
Nithya: BDD100K Model, Abstract, Conclusion
Victor: PCA/TSNE, Introduction, Generalizability
Matt: 2D/3D CNN Models, MoViNet Embedding, Data re-splitting, Features: Learned Features, Classification: Deep-model training and leakage controls, Classification: Held-out test results, Portions of Efficiency vs. Accuracy

References

Bradski, G. "The OpenCV Library." Dr. Dobb's Journal of Software Tools, 2000.

Carreira, J., Noland, E., Banki-Horvath, A., Hillier, C., and Zisserman, A. "A Short Note about Kinetics-600." arXiv:1808.01340, 2018.

Dalal, N. and Triggs, B. "Histograms of Oriented Gradients for Human Detection." CVPR 2005.

Ettinger, S., et al. "Large Scale Interactive Motion Forecasting for Autonomous Driving: The Waymo Open Motion Dataset." ICCV 2021.

Farnebäck, G. "Two-Frame Motion Estimation Based on Polynomial Expansion." Scandinavian Conference on Image Analysis, 2003.

He, K., Zhang, X., Ren, S., and Sun, J. "Deep Residual Learning for Image Recognition." CVPR 2016.

Jocher, G., Chaurasia, A., and Qiu, J. "Ultralytics YOLOv8." github.com/ultralytics/ultralytics, 2023.

Konev, S. "StandardE2E: A Unified Framework for End-to-End Autonomous Driving Datasets." arXiv:2606.04271, 2026.

Kondratyuk, D., et al. "MoViNets: Mobile Video Networks for Efficient Video Recognition." CVPR 2021.

Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research 12, 2011.

van der Maaten, L. and Hinton, G. "Visualizing Data using t-SNE." Journal of Machine Learning Research 9, 2008.

Xu, L., et al. "WOD-E2E: Waymo Open Dataset for End-to-End Driving in Challenging Long-tail Scenarios." CVPR 2026.

Yu, F., et al. "BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning." CVPR 2020.